Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado pede funções de sincronização para o embarque de passageiros nos vagões da METROREC, em que cada vagão e cada passageiro são threads independentes. Entrego a `struct station` e as funções `station_init`, `station_load_car`, `station_wait_for_car` e `station_on_board` em C, usando apenas um mutex e variáveis de condição do pthreads. A compilação é feita por Makefile, que gera o executável `sync_car`, e há um teste extra para entrada inválida (`test_invalid`).

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
C, executável em Linux/Unix/macOS	OK	make no WSL (Ubuntu); evidencias.log
Um único lock por struct station	OK	include/sync_car.h: um pthread_mutex_t
Usa só pthread_mutex_* e pthread_cond_*	OK	Leitura de src/sync_car.c
Sem semáforos nem outras primitivas	OK	grep sem_ src/sync_car.c vazio
Um carro na estação por vez	OK	Premissa do enunciado; o runner usa um carro por vez
Embarque simultâneo (vários wait_for_car retornam antes de on_board)	OK	Contador boardingPassengers; runner libera até 49 passageiros por carro
load_car não retorna antes do embarque	OK	Runner: sem "returned early"; print do Looks good!
load_car retorna na hora sem vagas ou sem passageiros	OK	2 primeiros testes do runner (alarm de 1–2 s)
Sem busy-wait	OK	Toda espera é pthread_cond_wait dentro de while
Makefile gera sync_car e limpa	OK	make clean && make; make clean
Caso inválido: count negativo	OK	./test_invalid: cenários 1 e 2 OK

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean && make
```

Executar (exemplo):

```
Executar o teste do professor (deve terminar com Looks good!):

./sync_car

Executar o teste de entrada inválida (dois cenários OK):

./test_invalid

Execução repetida (não imprime nada se as 100 passarem):

for i in $(seq 100); do ./sync_car > /dev/null || echo "falhou na execução $i"; done
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **include/sync_car.h → struct station (1 mutex, condições carAvailable e allBoarded, contadores waitingPassengers, freeSeats, boardingPassengers) e protótipos.**
2. **src/sync_car.c → implementação das quatro funções.**
3. **src/sync_car-runner.c → teste do professor, sem alterações.**
4. **src/test_invalid.c → teste de count negativo (estação vazia e com 3 passageiros esperando).**
5. **Makefile → gera sync_car e test_invalid com -Wall -Wextra -linclude -pthread; clean apaga os dois.**

Decisões técnicas:

1. **count < 0 vira 0; passageiro espera com freeSeats <= 0 → Entrada inválida → Ninguém embarca com contagem negativa**
2. **struct station definida no header → O runner declara struct station na pilha e só inclui sync_car.h → O .c inclui o header; a struct faz parte dele na compilação**
3. **while em volta de todo pthread_cond_wait → Spurious wakeups e disputa pela última vaga após o broadcast→ Ninguém embarca sem vaga (com if, freeSeats ficaria negativo)**
4. **Contador boardingPassengers -> Requisito 5: vários passageiros liberados antes de on_board ->O carro só parte quando todos os liberados sentaram**
5. **Zerar freeSeats na partida -> Vagas sobrando ficariam disponíveis após o carro sair -> Passageiro novo não embarca num carro que partiu**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	1 mutex + 2 condições + 3 contadores	Passageiros dormem em carAvailable, carro dorme em allBoarded; contadores de esperando, vagas e embarcando	
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Header com typedef struct station {...} station;	Parcial	Parâmetro station esconde o typedef; typo carAvailabe	~11:55	Commit 8c4fc96 (versão corrigida)

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
2	S1	Primeiro make	Falhou	gcc ausente no WSL (Error 127); resolvido com build-essential	~12:21	evidencias.log, linhas 48, 54, 59
3	S1	make com funções vazias	OK		~12:25	evidencias.log
4	S1	./sync_car com funções vazias	Falhou (esperado)	load_car vazia não espera: "returned early!"	~12:25	evidencias.log, l. 410–425; print
5	S1	station_init, wait_for_car, on_board	OK		17:15–17:42	Commits fae95fd, 64b3a57, 287e743
6	S1	station_load_car e ./sync_car	OK		~18:01	Print Looks good!; commit 39b7ea4
7	S1	100 execuções seguidas	OK	Parecia travado (~1 min): o runner espera ≥ 50 ms por carro	~18:03	Print; evidencias.log
8	S1	test_invalid.c, cenário 1	OK	VS Code marcava SIGALRM como indefinido (IntelliSense com headers do Windows)	~18:25	Print
9	S1	test_invalid.c, cenário 2	OK após correção	Array fora dos limites (&v[3]), %s com int, faltava string.h	~18:49	Print; commit a8b9710
10	S1	Alvo test_invalid no Makefile	OK após correção	Regra copiada gerava sync_car em vez de test_invalid	~18:57	evidencias.log; commit 0ab4beb

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

Erro principal: station_load_car returned early! com as funções vazias (evidencias.log, linhas 410–425)

```
theuz@Mth7:/mnt/c/mcr$ sed -n '410,425p' evidencias.log
src/sync_car-runner.c: In function ‘alarm_handler’:
src/sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
theuz@Mth7:/mnt/c/mcr$ ./sync_car
car entering station with 16 free seats
Error: station_load_car returned early!
theuz@Mth7:/mnt/c/mcr$ make clean   # apaga o executável
make clean && make   # recompila do zero, como pede o guia
rm -f sync_car
rm -f sync_car
theuz@Mth7:/mnt/c/mcr$
```

Validação final: ./sync_car terminando em Looks good!

```
Problems  Output  Debug Console  Terminal  Ports

theuz
/mnt/c/mcr
theuz@Mth7:/mnt/c/mcr$ ./sync_car
car entering station with 41 free seats
car departed station with 41 new passenger(s) (expected 41)
car entering station with 30 free seats
car departed station with 30 new passenger(s) (expected 30)
car entering station with 19 free seats
car departed station with 19 new passenger(s) (expected 19)
car entering station with 0 free seats
car departed station with 0 new passenger(s) (expected 0)
car entering station with 37 free seats
car departed station with 10 new passenger(s) (expected 10)
Looks good!
theuz@Mth7:/mnt/c/mcr$

powershell
bash
wsl
```

Execução repetida: 100 execuções sem falha

```
Looks good!
theuz@Mth7:/mnt/c/mcr$ for i in $(seq 100); do ./sync_car > /dev/null || echo "falhou na execução $i"; done
theuz@Mth7:/mnt/c/mcr$
```

Caso inválido: ./test_invalid com os dois cenários OK

```
theuz@Mth7:/mnt/c/mcr$ gcc -Wall -Wextra -Iinclude -pthread -o test_invalid src/sync_car.c src/test_invalid.c
./test_invalid
Cenário 1 (count negativo, estação vazia): OK
Cenário 2 (count negativo, passageiros esperando): OK
theuz@Mth7:/mnt/c/mcr$
```

7. Uso de IA

Onde usei IA: usei o Claude como tutor para entender o enunciado, planejar a struct e a lógica de cada função, organizar pastas, Makefile e commits, e revisar o que eu escrevia. O código de sync_car.h, sync_car.c, test_invalid.c e do Makefile foi digitado por mim a partir das explicações; o README e o rascunho deste relatório foram redigidos pela IA a partir da sessão e revisados por mim.

Prompts principais:

• Ajuda para construir a implementação a partir do enunciado e do runner

• Organização de pastas (src/, include/), .gitignore e mensagens de commit atômicas

• Revisão do header (typedef, typo) e do Makefile (alvos, TAB, header nas dependências)

• Explicações sobre pthread_cond_wait, while × if, signal × broadcast

• Revisão de cada função e da condição de partida do carro

• Teste de entrada inválida (pedi o código pronto; a IA recusou e passou os passos)

• Montagem do README e deste relatório

O que validei manualmente e como: compilei com make clean && make sem warnings no meu código; rodei ./sync_car (Looks good!), 100 execuções seguidas sem falha e ./test_invalid (dois cenários OK), tudo gravado no evidencias.log. A IA também compilou e rodou cópias do código no ambiente dela para revisar, sem alterar meus arquivos.

8. Reflexão Final (5–8 linhas)

Durante a criação do projeto eu aprendi a importância da sincronização de threads. A decisão técnica central foi separar “esperando” de “embarcando”, sem o terceiro contador o carro não saberia quando todos os liberados sentaram. No início tive dificuldade em entender como implementar essa lógica, porém testando e validando com a IA, consegui desenvolver. Erros que eu evitaria seriam, começar sem verificar o gcc, e sem ligar o log, além disso, evitaria erros como “off-by-one” no array de threads e copiar uma regra do makefile sem alterar as variáveis.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Se eu tivesse mais duas horas, eu focaria em aprimoração do código. Implementado certas mudanças como a troca do fprintf do tratador de sinal do teste por write/_exit. Além disso, também testaria os retornos das funções de forma robusta e testaria o requisito 5 diretamente um teste que confirme vários wait_for_car retornando antes de qualquer on_board. No geral, seria isso que eu focaria em trocar ou analisar se a troca é eficaz nas duas horas adicionais.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.